2027-7-24 Linux库--动静态库的使用

- 1. 识别库与程序依赖
  - 1.1 动静态库的基本区别
    - 1.1.1 两个平台中的库文件
    - 1.1.2 用 ldd 查看实际依赖
  - 1.2 从源代码到库文件
    - 1.2.1 先把每个源文件编译成 .o
    - 1.2.2 库文件和头文件要一起交付
- 2. 静态库的生成与使用
  - 2.1 生成静态库
    - 2.1.1 用 ar 打包 .o
    - 2.1.2 整理发布目录
  - 2.2 链接静态库
    - 2.2.1 安装到系统目录
    - 2.2.2 用 -I、-L、-l 指定搜索位置
    - 2.2.3 通过软链接接入系统搜索路径
- 3. 动态库的生成、链接与查找
  - 3.1 生成动态库
    - 3.1.1 编译位置无关代码并生成共享库
  - 3.2 链接动态库与运行时查找
    - 3.2.1 编译成功不等于运行时能找到动态库
  - 3.3 解决动态库运行时找不到的问题
    - 3.3.1 复制到系统共享库目录
    - 3.3.2 在系统共享库目录建立软链接
    - 3.3.3 使用 LD_LIBRARY_PATH
    - 3.3.4 使用 ldconfig 持久化搜索路径
    - 3.3.5 复查依赖是否已经解析
- 4. 动静态库的选择与第三方库
  - 4.1 静态库和动态库的链接优先级
    - 4.1.1 同名库同时存在时，默认优先动态库
    - 4.1.2 用移除库文件的方式理解区别
  - 4.2 用同一套规则调用第三方库
    - 4.2.1 安装头文件、库文件并指定库名

1. 识别库与程序依赖

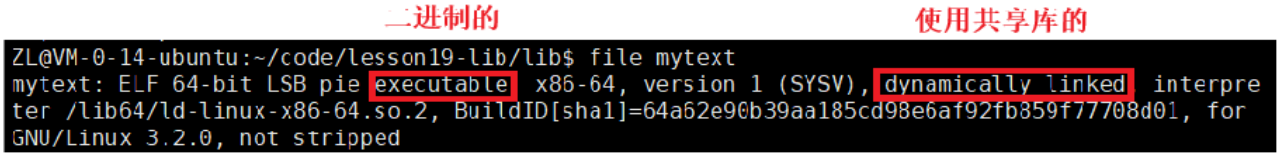
1.1 动静态库的基本区别

1.1.1 两个平台中的库文件

Linux 和 Windows 对静态库、动态库使用不同的文件后缀：

平台	静态库	动态库
Linux	`.a`	`.so`
Windows	`.lib`	`.dll`

使用 `gcc`、`g++` 编译链接 C/C++ 程序时，默认优先采用动态链接。



1.1.2 用 `ldd` 查看实际依赖

`ldd` 用于查看可执行文件或共享库依赖了哪些库。例如：

```
ldd ./mytest
```

输出中的 `libc.so.6 => /lib/.../libc.so.6` 表示程序依赖 C 标准库。Linux 下的库通常会在 `/lib`、`/lib64`、`/usr/lib` 或 `/usr/local/lib` 等目录中查找。

```
ZL@VM-0-14-ubuntu:~/code/lesson19-lib/lib$ vim text.c
ZL@VM-0-14-ubuntu:~/code/lesson19-lib/lib$ gcc text.c -o mytext
ZL@VM-0-14-ubuntu:~/code/lesson19-lib/lib$ ldd mytext
        linux-vdso.so.1 (0x00007ffe9dd98000)
        libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007aaa12800000)
        /lib64/ld-linux-x86-64.so.2 (0x00007aaa12b94000)
ZL@VM-0-14-ubuntu:~/code/lesson19-lib/lib$
```

```
ZL@VM-0-14-ubuntu:~/code/lesson19-lib/lib$ ls -l /usr/lib/x86_64-linux-gnu/libc.so.6
-rwxr-xr-x 1 root root 2125328 Jan 30 16:27 /usr/lib/x86_64-linux-gnu/libc.so.6
```

1.2 从源代码到库文件

1.2.1 先把每个源文件编译成 `.o`

编译多个源文件时，更清晰的过程是：

```
gcc -c main.c
gcc -c mystdio.c
gcc -c mystring.c
gcc -o target main.o mystdio.o mystring.o
```

`gcc -c` 只编译、不链接，因此可以单独得到 `.o` 目标文件。多个 `.o` 只有在链接阶段才会发生关联。最终把全部 `.c` 文件一起编译、链接虽然也能生成程序，但不利于复用和局部重新编译。

```
ZL@VM-0-14-ubuntu:~/code/lesson19-lib/lib/stdio$ gcc -o target main.c mystdio.c mystring.c
ZL@VM-0-14-ubuntu:~/code/lesson19-lib/lib/stdio$ ./target
debug:outbuffer =  hello,pos = 5
debug:outbuffer =  hellohello,pos = 10
debug:outbuffer =  hellohellohello,pos = 15
debug:outbuffer =  hellohellohellohello,pos = 20
debug:outbuffer =  hellohellohellohellohello,pos = 25
debug:outbuffer =  hellohellohellohellohellohello,pos = 30
debug:outbuffer =  hellohellohellohellohellohellohello,pos = 35
debug:outbuffer =  hellohellohellohellohellohellohellohello,pos = 40
debug:outbuffer =  hellohellohellohellohellohellohellohellohello,pos = 45
debug:outbuffer =  hellohellohellohellohellohellohellohellohellohello,pos = 50
这是我自己的程序ZL@VM-0-14-ubuntu:~/code/lesson19-lib/lib/stdio$ vim main.c
```

```
ZL@VM-0-14-ubuntu:~/code/lesson19-lib/lib/stdio$ gcc -c mystring.c -o mystring.o
ZL@VM-0-14-ubuntu:~/code/lesson19-lib/lib/stdio$ gcc -o target *.o
ZL@VM-0-14-ubuntu:~/code/lesson19-lib/lib/stdio$ ll
total 64
drwxrwxr-x 2 ZL ZL  4096 Jul 25 10:33 ./
drwxrwxr-x 4 ZL ZL  4096 Jul 25 03:02 ../
-rw-rw-r-- 1 ZL ZL     0 Jul 25 10:28 log.txt
-rw-rw-r-- 1 ZL ZL   395 Jul 25 10:27 main.c
-rw-rw-r-- 1 ZL ZL  1728 Jul 25 10:31 main.o
-rw-rw-r-- 1 ZL ZL    73 Jul 24 22:18 Makefile
-rw-rw-r-- 1 ZL ZL  2391 Jul 24 22:19 mystdio.c
-rw-rw-r-- 1 ZL ZL   532 Jul 24 22:19 mystdio.h
-rw-rw-r-- 1 ZL ZL  3512 Jul 25 10:31 mystdio.o
-rw-rw-r-- 1 ZL ZL    79 Jul 25 10:28 mystring.c
-rw-rw-r-- 1 ZL ZL    49 Jul 25 10:24 mystring.h
-rw-rw-r-- 1 ZL ZL  1520 Jul 25 10:32 mystring.o
-rwxrwxr-x 1 ZL ZL 16608 Jul 25 10:33 target*
```

可以使用 `Makefile` 固定这套流程：

```
target: main.o mystdio.o mystring.o
    gcc -o $@ $^

%.o: %.c
    gcc -c $<
```

```
.PHONY: clean
clean:
    rm -f *.o target
```

这样修改某个 `.c` 文件时，只需要重新编译对应的 `.o`，再把全部 `.o` 链接起来。

```
ZL@VM-0-14-ubuntu:~/code/lesson19-lib/lib/stdio$ ll
total 32
drwxrwxr-x 2 ZL ZL 4096 Jul 25 10:54 ./
drwxrwxr-x 4 ZL ZL 4096 Jul 25 03:02 ../
-rw-rw-r-- 1 ZL ZL    0 Jul 25 10:33 log.txt
-rw-rw-r-- 1 ZL ZL  395 Jul 25 10:27 main.c
-rw-rw-r-- 1 ZL ZL  108 Jul 25 10:54 Makefile
-rw-rw-r-- 1 ZL ZL 2391 Jul 24 22:19 mystdio.c
-rw-rw-r-- 1 ZL ZL  532 Jul 24 22:19 mystdio.h
-rw-rw-r-- 1 ZL ZL   79 Jul 25 10:28 mystring.c
-rw-rw-r-- 1 ZL ZL   49 Jul 25 10:24 mystring.h
ZL@VM-0-14-ubuntu:~/code/lesson19-lib/lib/stdio$ make
gcc -c main.c
gcc -c mystdio.c
gcc -c mystring.c
gcc -o target main.o mystdio.o mystring.o
```

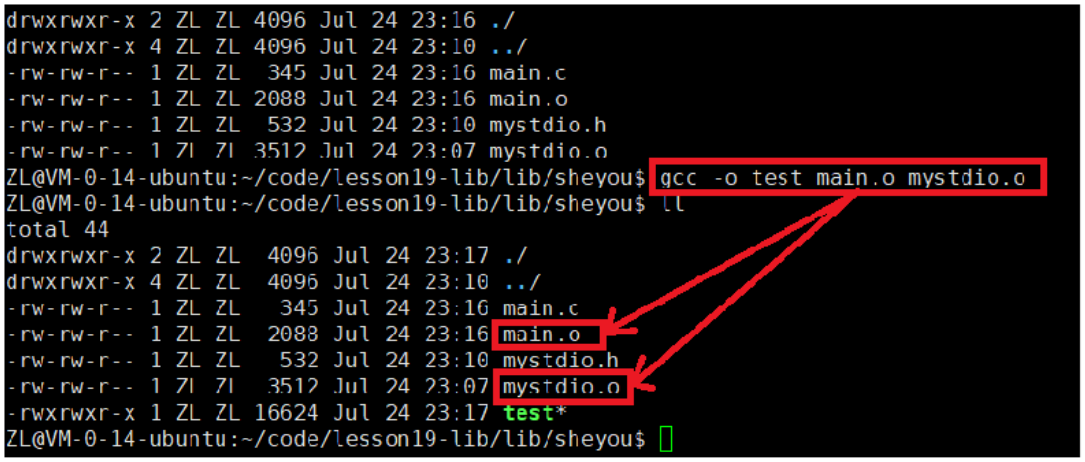
1.2.2 库文件和头文件要一起交付

库可以看作一批可复用 `.o` 文件的集合。交付给调用者时，至少需要：

```
mylib/
├── include/
│   └── mystdio.h
└── lib/
    └── libmystdio.a   # 或 libmystdio.so
```

头文件相当于库的使用说明，声明调用者可以使用的函数和类型；库文件提供这些函数的实现。库中不能包含 `main` 函数，因为真正的主函数应由程序的使用者提供。

以我们之前封装的系统调用为例，有下面场景：
有一天，你的老师布置了一个封装系统调用的作业，你的同学小李写的抓耳挠腮的于是乎向你求助，
你说：“不行，我不能把我的源文件给你，万一老师先看了你的代码说我抄你的代码怎么办？”
小李：“好兄弟，但是你不能见死不救啊！”
这个时候你说，行吧我编成.c文件给你，再把方法给你直接用就好。这个时候小李就只需要将自己的main函数编译成.o文件，再把其他文件连接到一起就行。你给的.o文件本质是这个文件的半成品，实际上是不是和库差不多？ 库的本质就是.o文件的集合



2. 静态库的生成与使用

2.1 生成静态库

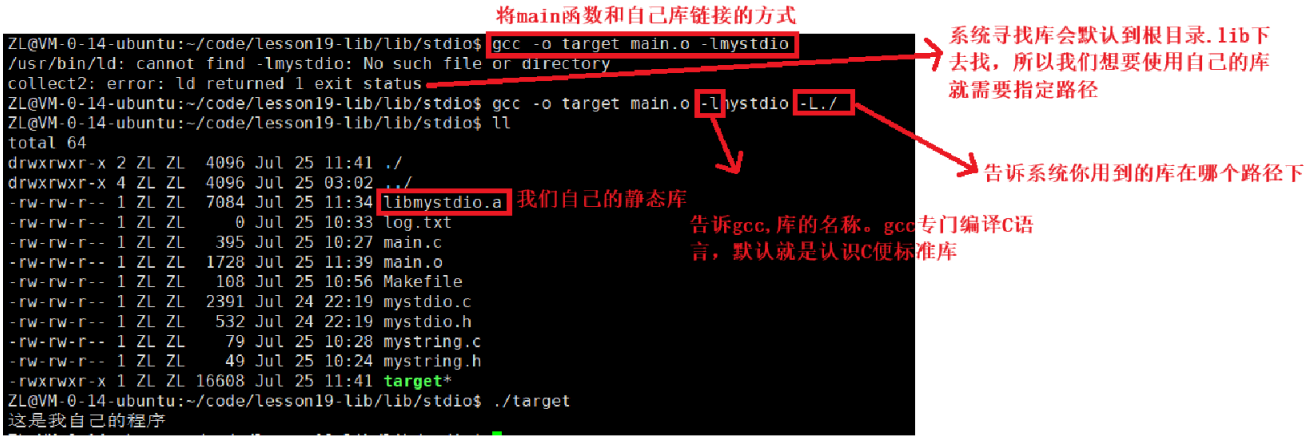
2.1.1 用 `ar` 打包 `.o`

先把源文件编译为位置无关性要求较低的普通目标文件，再用 `ar` 归档：

```
gcc -c mystdio.c
gcc -c mystring.c
ar -rc libmystdio.a mystdio.o mystring.o
```

`ar` 是 GNU 归档工具，`rc` 表示 replace and create。可以检查静态库中包含哪些目标文件：

```
ar -tv libmystdio.a
```



对应的 `Makefile` 可以写成：

```
libmystdio.a: mystdio.o mystring.o
    ar -rc $@ $^

%.o: %.c
    gcc -c $<

.PHONY: clean
clean:
    rm -f *.o *.a
```

2.1.2 整理发布目录

发布静态库时，把头文件和 `.a` 文件分别放入 `include`、`lib`：

```
.PHONY: output
output:
    mkdir -p mylib/include mylib/lib
    cp *.h mylib/include
    cp *.a mylib/lib
```

调用者拿到这两个目录后，就可以开始编译自己的程序。

```
ZL@VM-0-14-ubuntu:~/code/lesson19-lib/lib/stdio$ make output
mkdir -p mylib
mkdir -p mylib/include
mkdir -p mylib/lib
cp *.h mylib/include
cp *.a mylib/lib
cp: cannot stat '*.a': No such file or directory
make: *** [Makefile:14: output] Error 1
ZL@VM-0-14-ubuntu:~/code/lesson19-lib/lib/stdio$ tree ./
./
├── log.txt
├── main.c
├── Makefile
├── mylib
│   ├── include
│   │   ├── mystdio.h
│   │   └── mystring.h
│   └── lib
│       └── mylibstdio.a
├── mystdio.c
├── mystdio.h
├── mystring.c
├── mystring.h
└── target

4 directories, 11 files
ZL@VM-0-14-ubuntu:~/code/lesson19-lib/lib/stdio$
```

2.2 链接静态库

2.2.1 安装到系统目录

第一种方式是把头文件和静态库复制到系统搜索路径：

```
sudo cp mylib/include/* /usr/include/
sudo cp mylib/lib/libmystdio.a /usr/lib/
```

之后可以直接按库名链接：

```
gcc main.c -o main -lmystdio
```

```
apparmor          cryptsetup        git-core          klibc                                  lsb               NetworkManager  python3.12        tmpfiles.d
apt               dbus-1.0          gnupg             klibc-sw0VayLfV0hmLGCQE9vyfOnj81g.so    man-db            nvidia          recovery-mode     ubiquity
bfd-plugins       debug             gnupg2            libmpathcmd.so.0                       mime              open-iscsi      rsyslog           ubuntu-advantage
binfmt.d          dhcpcd            gold-ld           libmpathpersist.so.0                   modprobe.d        openssh         sasl2             ubuntu-release-upgrade
byobu             dpkg              groff             libmpathutil.so.0                      modules           os-probes       sftp-server       udev
cloud-init        dracut            grub              libmultipath.so.0                      modules-load.d    os-release      snapd             udisks2
cnf-update-db     environment.d     grub-legacy       linux                                  multipath         pam.d           software-properties ufw
command-not-found file              hdparm            linux-boot-probes                      mylibstdio.a      pcrlock.d       ssl               update-notifier
compat-ld         finalrd           init              linux-tools                            nagios            pm-utils        sysctl.d          valgrind
console-setup     firmware          initcpio          linux-tools-6.8.0-117                  needrestart       policykit-1     sysstat           x86_64-linux-gnu
cpp               gcc               initramfs-tools   llvm-18                                netplan           polkit-1        systemd
crash             girepository-1.0  kernel            locale                                 networkd-dispatcher python3       sysusers.d
```

```
ZL@VM-0-14-ubuntu:~/code/lesson19-lib/lib/stdio$ gcc main.c -lmystdio
ZL@VM-0-14-ubuntu:~/code/lesson19-lib/lib/stdio$ ./a.out
这是我自己的程序
ZL@VM-0-14-ubuntu:~/code/lesson19-lib/lib/stdio$ ll
total 92
drwxrwxr-x 3 ZL ZL  4096 Jul 25 14:38 ./
drwxrwxr-x 4 ZL ZL  4096 Jul 25 03:02 ../
-rwxrwxr-x 1 ZL ZL 16608 Jul 25 14:38 a.out*
-rw-rw-r-- 1 ZL ZL  5288 Jul 25 12:34 libmystdio.a
-rw-rw-r-- 1 ZL ZL     0 Jul 25 14:38 log.txt
-rw-rw-r-- 1 ZL ZL   395 Jul 25 10:27 main.c
-rw-rw-r-- 1 ZL ZL   233 Jul 25 12:29 Makefile
drwxrwxr-x 4 ZL ZL  4096 Jul 25 12:24 mylib/
-rw-rw-r-- 1 ZL ZL  2391 Jul 24 22:19 mystdio.c
-rw-rw-r-- 1 ZL ZL   532 Jul 24 22:19 mystdio.h
-rw-rw-r-- 1 ZL ZL  3512 Jul 25 12:34 mystdio.o
-rw-rw-r-- 1 ZL ZL    79 Jul 25 10:28 mystring.c
-rw-rw-r-- 1 ZL ZL    49 Jul 25 10:24 mystring.h
-rw-rw-r-- 1 ZL ZL  1520 Jul 25 12:34 mystring.o
-rwxrwxr-x 1 ZL ZL 16608 Jul 25 11:41 target*
ZL@VM-0-14-ubuntu:~/code/lesson19-lib/lib/stdio$
```

2.2.2 用 `-I`、`-L`、`-l` 指定搜索位置

不修改系统目录时，应显式告诉编译器头文件和库文件的位置：

```
gcc main.c -o main \
    -I ./mylib/include \
    -L ./mylib/lib \
    -lmystdio
```

三个选项分别表示：

选项	作用
`-I`	增加头文件搜索目录
`-L`	增加库文件搜索目录
`-l`	指定库名

链接器识别的是去掉 `lib` 前缀和 `.a`、`.so` 后缀后的名字。例如 `libmystdio.a` 应写成 `-lmystdio`。

如果只指定了头文件目录，而没有编译库文件，那么链接阶段会报告函数未定义；如果头文件目录没有指定正确，则编译器会找不到对应头文件。

```
ZL@VM-0-14-ubuntu:~/code/lesson19-lib/lib/stdio$ gcc -c main.c
main.c:1:9: fatal error: mystdio.h: No such file or directory
    1 | #include"mystdio.h"
      |         ^~~~~~~~~~~
compilation terminated.
ZL@VM-0-14-ubuntu:~/code/lesson19-lib/lib/stdio$
```

2.2.3 通过软链接接入系统搜索路径

也可以把头文件安装到系统头文件目录，再为静态库建立同名软链接：

```
sudo ln -s "$(realpath ./mylib/lib/libmystdio.a)" /usr/lib/libmystdio.a
gcc main.c -o main -lmystdio
```

这种方式仍属于“安装到系统路径”，只是库文件本体保留在原位置。

```
ZL@VM-0-14-ubuntu:~/code/lesson19-lib/lib/stdio$ sudo ln -s /home/ZL/code/lesson19-lib/lib/stdio/mylib/lib/libmystdio.a /lib64/libmystdio.a
[sudo] password for ZL:
ZL@VM-0-14-ubuntu:~/code/lesson19-lib/lib/stdio$ ls /lib64/libmystdio.a -l
lrwxrwxrwx 1 root root 59 Jul 25 15:51 /lib64/libmystdio.a -> /home/ZL/code/lesson19-lib/lib/stdio/mylib/lib/libmystdio.a
ZL@VM-0-14-ubuntu:~/code/lesson19-lib/lib/stdio$ sudo cp mylib/include/*.h /usr/include/
ZL@VM-0-14-ubuntu:~/code/lesson19-lib/lib/stdio$ gcc main -o main -lmaystdio
```

静态库在链接时会把程序实际使用到的目标代码复制到可执行文件中。因此，程序生成之后即使删除原 `.a` 文件，程序仍然可以运行。

3. 动态库的生成、链接与查找

3.1 生成动态库

3.1.1 编译位置无关代码并生成共享库

动态库会被映射到不同进程的地址空间，编译目标文件时需要 `-fPIC`：

```
gcc -fPIC -c mystdio.c
gcc -fPIC -c mystring.c
gcc -shared -o libmystdio.so mystdio.o mystring.o
```

其中：

选项	作用
`-fPIC`	生成位置无关代码，使共享库可以加载到任意地址
`-shared`	生成共享库

对应的 `Makefile`：

```
libmystdio.so: mystdio.o mystring.o
    gcc -shared -o $@ $^

%.o: %.c
    gcc -fPIC -c $<

.PHONY: clean
clean:
    rm -f *.o *.so
```

动态库仍然和头文件配套交付，目录结构与静态库相同，只是 `lib` 目录中放的是 `libxxx.so`。

3.2 链接动态库与运行时查找

3.2.1 编译成功不等于运行时能找到动态库

链接动态库的编译命令与静态库相同：

```
gcc main.c -o main \
    -I ./mylib/include \
    -L ./mylib/lib \
    -lmystdio
```

这条命令只把动态库的路径和库名交给编译器。可执行文件生成后，真正加载动态库的是操作系统加载器和动态链接器。

常见现象是编译通过，但运行失败：

```
./main: error while loading shared libraries:
libmystdio.so: cannot open shared object file: No such file or directory
```

此时查看依赖：

```
ldd ./main
```

可能会看到：

```
libmystdio.so => not found
```

原因不是编译器没有找到库，而是程序运行时，动态链接器没有在默认搜索路径中找到 `libmystdio.so`。

```
ZL@VM-0-14-ubuntu:~/code/lesson19-lib/lib/stdio$ ./main
./main: error while loading shared libraries: libmystdio.so: cannot open shared object file:
No such file or directory
ZL@VM-0-14-ubuntu:~/code/lesson19-lib/lib/stdio$ ldd main
        linux-vdso.so.1 (0x00007ffca4faa000)
        libmystdio.so => not found
        libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007332df400000)
        /lib64/ld-linux-x86-64.so.2 (0x00007332df724000)
ZL@VM-0-14-ubuntu:~/code/lesson19-lib/lib/stdio$
```

3.3 解决动态库运行时找不到的问题

3.3.1 复制到系统共享库目录

可以直接把 `.so` 放入 `/usr/lib`、`/usr/local/lib`、`/lib64` 等共享库目录：

```
sudo cp mylib/lib/libmystdio.so /usr/lib/
```

这种方式改动系统库目录，所有程序都可能受到影响，适用于需要全局安装的库。

3.3.2 在系统共享库目录建立软链接

如果希望库文件本体继续留在项目目录中，可以建立软链接：

```
sudo ln -s \
    "$(realpath ./mylib/lib/libmystdio.so)" \
    /usr/lib/libmystdio.so
```

```
ZL@VM-0-14-ubuntu:~/code/lesson19-lib/lib/stdio$ sudo ln -s /home/ZL/code/lesson19-lib/lib/stdio/mylib/lib/libmystdio.
so /usr/lib/libmystdio.so
ZL@VM-0-14-ubuntu:~/code/lesson19-lib/lib/stdio$ ll /usr/lib/libmystdio.so
lrwxrwxrwx 1 root root 60 Jul 25 18:36 /usr/lib/libmystdio.so -> /home/ZL/code/lesson19-lib/lib/stdio/mylib/lib/libmys
tdio.so*
ZL@VM-0-14-ubuntu:~/code/lesson19-lib/lib/stdio$ ldd main
        linux-vdso.so.1 (0x00007ffd98d39000)
        libmystdio.so => /lib/libmystdio.so (0x0000793cd7d07000)
        libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x0000793cd7a00000)
        /lib64/ld-linux-x86-64.so.2 (0x0000793cd7d1a000)
ZL@VM-0-14-ubuntu:~/code/lesson19-lib/lib/stdio$ ./main
这是我自己的程序
ZL@VM-0-14-ubuntu:~/code/lesson19-lib/lib/stdio$
```

3.3.3 使用 `LD_LIBRARY_PATH`

只在当前 shell 会话中增加动态库搜索目录：

```
export LD_LIBRARY_PATH="$PWD/mylib/lib:${LD_LIBRARY_PATH}"
./main
```

新开终端后该设置会失效。它适合临时测试，不建议把不可信目录长期放入动态库搜索路径。

```
ZL@VM-0-14-ubuntu:~/code/lesson19-lib/lib/stdio$ export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/home/ZL/code/lesson19-lib/lib/stdio/m
lib/lib
ZL@VM-0-14-ubuntu:~/code/lesson19-lib/lib/stdio$ echo $LD_LIBRARY_PATH
:/home/ZL/code/lesson19-lib/lib/stdio/mylib/lib:/home/ZL/code/lesson19-lib/lib/stdio/mylib/lib
ZL@VM-0-14-ubuntu:~/code/lesson19-lib/lib/stdio$ ldd main
        linux-vdso.so.1 (0x00007ffcf058c000)
        libmystdio.so (0x000074176a168000)
        libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x0000741769e00000)
        /lib64/ld-linux-x86-64.so.2 (0x000074176a174000)
ZL@VM-0-14-ubuntu:~/code/lesson19-lib/lib/stdio$ ./main
这是我自己的程序
ZL@VM-0-14-ubuntu:~/code/lesson19-lib/lib/stdio$
```

3.3.4 使用 `ldconfig` 持久化搜索路径

可以把库所在目录写入 `/etc/ld.so.conf.d/` 下的配置文件，再刷新动态链接器缓存：

```
echo "$PWD/mylib/lib" | sudo tee /etc/ld.so.conf.d/mylib.conf
sudo ldconfig
ldconfig -p | grep mystdio
```

执行 `ldconfig` 后，系统会重新生成缓存，后续程序就能从配置目录中找到该动态库。

```
ZL@VM-0-14-ubuntu:~/code/lesson19-lib/lib/stdio$ ls /etc/ld.so.conf
/etc/ld.so.conf
ZL@VM-0-14-ubuntu:~/code/lesson19-lib/lib/stdio$ ls /etc/ld.so.conf.d/
libc.conf  x86_64-linux-gnu.conf
ZL@VM-0-14-ubuntu:~/code/lesson19-lib/lib/stdio$
```

3.3.5 复查依赖是否已经解析

每次处理完后都重新检查：

```
ldd ./main
./main
```

只有 `libmystdio.so` 不再显示 `not found`，并且程序能够正常启动，动态库的使用才算真正完成。

4. 动静态库的选择与第三方库

4.1 静态库和动态库的链接优先级

4.1.1 同名库同时存在时，默认优先动态库

当搜索路径中同时存在 `libmystdio.a` 和 `libmystdio.so` 时，`gcc` 默认优先链接动态库。

提供的形式	默认结果	运行时是否还需要原库
同时提供 `.a` 和 `.so`	优先动态链接	需要 `.so`
只提供 `.a`	静态链接	不需要 `.a`
只提供 `.so`	动态链接	需要 `.so`

如果确实需要静态链接，可以使用 `-static`：

```
gcc main.c -o main_static \
    -I ./mylib/include \
    -L ./mylib/lib \
    -lmystdio \
    -static
```

`-static` 必须谨慎使用：它不仅要求当前库存在 `.a` 版本，还可能要求系统依赖的 C 库等也具备静态版本。

4.1.2 用移除库文件的方式理解区别

静态程序生成后，可以暂时移走静态库再运行；动态程序移走 `.so` 后通常会启动失败：

```
mv mylib/lib/libmystdio.a /tmp/
./main_static

mv mylib/lib/libmystdio.so /tmp/
./main
```

这组对比说明了核心差异：

- 静态链接在程序生成阶段复制所需代码，程序运行时不再依赖原 `.a`。
- 动态链接在程序启动时加载共享库，程序运行时仍然需要找到原 `.so`。
- 只提供动态库时，无法通过 `-static` 强行得到该库的静态版本。

4.2 用同一套规则调用第三方库

4.2.1 安装头文件、库文件并指定库名

第三方库通常同样分为“安装开发包”和“编译时链接”两步。以 `ncurses` 为例：

```
sudo apt install libncurses-dev
gcc main.c -std=c99 -lncurses -lm
```

如果系统头文件和库文件已经位于标准搜索路径，直接使用 `-lncurses` 即可；如果它们位于自定义目录，仍然使用 `-I`、`-L`、`-l` 指定头文件目录、库目录和库名。若第三方库以动态库形式提供，运行前也仍需保证动态链接器能够找到对应的 `.so`。